CS285 Homework 5

Exploration Strategies and Offline Reinforcement Learning

杨丹茹

摘要

本报告围绕 Berkeley CS285 Homework 5 展开，完整覆盖理论分析、探索数据收集、离线强化学习、数据规模消融和离线到在线微调五部分任务。首先推导 KL-divergence、一般 f-divergence 以及状态轨迹分布 KL 约束下的 reward bonus；随后实现 Random Policy、Random Network Distillation（RND）、DQN、CQL、AWAC、IQL 及 online fine-tuning 的核心代码，并按照作业顺序给出关键 TODO 的逐行说明。实验结果表明：RND 相比随机策略显著扩大了状态覆盖；CQL 的保守系数并非越大越好，$\alpha = 0.1$ 在 PointmassMedium-v0 上取得较好折中；在 PointmassHard-v0 的离线实验中，IQL 的最佳和最终 evaluation return 均优于 AWAC；Data Ablation 显示 20000-step 数据集获得最高且最稳定的表现，而 1000-step 数据集虽然出现较高的 mean Q-value，却对应很差的真实回报，说明 Q-value 的绝对大小不能代替真实评估。Online fine-tuning 明显改善了策略和状态覆盖，但当前实验仍主要稳定在 -30 附近，尚未达到题目期望的稳定 -20，因此本报告对该性能差距及其原因进行了如实分析。

目录

1 作业整体说明与要求对照 **2**
1.1 作业要求覆盖检查 . . . 2

2 Analysis 理论题 **2**
2.1 KL-divergence 约束下的 reward bonus . . . 2
2.2 f-divergence 约束下的 reward bonus . . . 3
2.3 状态轨迹分布 KL 约束下的 reward bonus . . . 3

3 Coding 总体实现 **4**

4 Exploration 实验 **4**
4.1 Random Policy . . . 4
4.1.1 题目要求与实现 . . . 4
4.1.2 实验图与分析 . . . 4
4.2 Random Network Distillation . . . 5
4.2.1 算法原理 . . . 5
4.2.2 RND predictor 与 intrinsic reward . . . 5
4.2.3 当前代码中的 exploration critic 等价实现 . . . 6
4.2.4 实验结果 . . . 6

5 Offline RL：DQN 与 CQL **7**
5.1 DQN / Double DQN . . . 7
5.1.1 动作选择 . . . 7

5.1.2 critic loss . . . 7
5.2 Conservative Q-Learning . . . 8
5.3 CQL α sweep 实验 . . . 9

6 Policy Constraint Methods：AWAC 与 IQL **10**
6.1 AWAC . . . 10
6.1.1 AWAC critic backup . . . 10
6.1.2 AWAC advantage 与 actor update . . . 10
6.2 Implicit Q-Learning . . . 11
6.2.1 Expectile loss . . . 11
6.2.2 Q-function 更新 . . . 11
6.2.3 Value function 更新 . . . 12
6.3 AWAC 与 IQL 实验对比 . . . 13

7 Data Ablation **13**
7.1 实验设计 . . . 13
7.2 结果分析 . . . 15

8 Online Fine-Tuning **15**
8.1 题目要求 . . . 15
8.2 核心实现 . . . 15
8.2.1 加载并扩容离线 replay buffer . . . 15
8.2.2 offline/online 阶段切换与参数调整 . . . 16
8.2.3 在线交互与持续训练 . . . 16
8.3 实验结果 . . . 17
8.4 未稳定达到 -20 的原因 . . . 18

9 总结与个人思考 **19**

10 提交说明 **19**

参考文献 **19**

1 作业整体说明与要求对照

本次作业的主题是 *Exploration Strategies and Offline Reinforcement Learning*。它要求实现一个完整流程：先使用随机策略与 RND 收集离线数据，再利用 DQN、CQL、AWAC 和 IQL 从固定数据集中学习策略，之后改变数据规模进行消融，最后实现 offline-to-online fine-tuning。

离线强化学习的核心困难是 **distribution shift**：训练阶段只能访问离线数据集，若学习策略选择了数据中很少出现或完全未出现的动作，Q 函数可能对这些 out-of-distribution（OOD）动作产生不可靠的高估。Homework 5 通过探索覆盖、保守价值学习、策略约束与在线补充数据，让我们从多个角度理解这一问题。

1.1 作业要求覆盖检查

作业部分	本报告中的对应内容	状态
Analysis	KL、f-divergence、状态轨迹 KL 三道理论题及推导	已完成
Random Policy	`RandomAgent.get_action`，Easy/Medium/Hard 覆盖图	已完成
RND	predictor/target、bonus、归一化、exploration critic 等价实现与 error map	已完成
DQN/CQL	DQN/Double DQN、CQL regularizer、α sweep、Q-value 与 eval 曲线	已完成
AWAC/IQL	critic、advantage、actor、expectile、Q/V 更新及对比曲线	已完成
Data Ablation	1000/5000/10000/20000-step 数据集对应的 CQL 曲线	已完成
Online Fine-Tuning	offline/online 切换、buffer 初始化、在线采样及结果图	流程完成；性能未达 -20
提交结构	PDF、`cs285/`、`data/`、README 要求说明	已覆盖

表 1: Homework 5 要求与本报告内容的对应关系。

本作业的主线是理解 “数据覆盖 → 价值估计可靠性 → 策略约束 → 在线修正” 的完整链条。探索数据若没有覆盖关键通道，后续算法无法凭空恢复成功轨迹；Q 函数过度乐观会选择 OOD 动作，过度保守又会压制有潜力的动作；online fine-tuning 能补充数据，但其效果仍取决于初始策略和 replay buffer 的样本质量。

2 Analysis 理论题

2.1 KL-divergence 约束下的 reward bonus

离线数据集 $\mathcal{D}$ 由行为策略 π_β 收集。我们希望学习策略 π，同时限制其偏离行为策略：

$$D(\pi, \pi_\beta) = \mathbb{E}_{s\sim p^\pi} D_{\mathrm{KL}}[\pi(a \mid s) \| \pi_\beta(a \mid s)] \,. \tag{1}$$

展开 KL 散度：

$$D_{\mathrm{KL}}[\pi \| \pi_\beta] = \mathbb{E}_{a\sim\pi}\left[\log \frac{\pi(a \mid s)}{\pi_\beta(a \mid s)}\right]. \tag{2}$$

为了在 $\bar{r}(s,a) = r(s,a) + \lambda b(s,a)$ 中形成负的 KL 惩罚，可取：

$$\boxed{b(s,a) = -\log \frac{\pi(a \mid s)}{\pi_\beta(a \mid s)} = \log \pi_\beta(a \mid s) - \log \pi(a \mid s).} \tag{3}$$

当某个动作在当前策略下概率很大、在行为策略下概率很小时，bonus 为较大负数，从而阻止策略过度偏离离线数据支持范围。

2.2 f-divergence 约束下的 reward bonus

一般 f-divergence 为：

$$D_f[P\|Q] = \int Q(x) f\left(\frac{P(x)}{Q(x)}\right) dx. \tag{4}$$

策略约束写成：

$$D(\pi, \pi_\beta) = \mathbb{E}_{s\sim p^\pi} \mathbb{E}_{a\sim\pi_\beta} \left[f\left(\frac{\pi(a \mid s)}{\pi_\beta(a \mid s)}\right)\right]. \tag{5}$$

把 $a \sim \pi_\beta$ 的期望改写为 $a \sim \pi$：

$$\mathbb{E}_{a\sim\pi_\beta}[f(\rho)] = \mathbb{E}_{a\sim\pi} \left[\frac{\pi_\beta(a \mid s)}{\pi(a \mid s)} f(\rho)\right], \qquad \rho = \frac{\pi(a \mid s)}{\pi_\beta(a \mid s)}. \tag{6}$$

因此可以选择：

$$\boxed{b(s,a) = -\frac{\pi_\beta(a \mid s)}{\pi(a \mid s)} f\left(\frac{\pi(a \mid s)}{\pi_\beta(a \mid s)}\right).} \tag{7}$$

当 $f(x) = x \log x$ 时恢复 KL 情况；选择其他凸函数可得到 TVD、JSD 等不同约束。

2.3 状态轨迹分布 KL 约束下的 reward bonus

有限时域状态轨迹为 $\tau = (s_1, \ldots, s_H)$。策略诱导的一步状态转移为：

$$p^\pi(s' \mid s) = \sum_a \pi(a \mid s) p(s' \mid s, a), \qquad p^{\pi_\beta}(s' \mid s) = \sum_a \pi_\beta(a \mid s) p(s' \mid s, a). \tag{8}$$

若允许 transition-dependent bonus，可以取：

$$b(s, a, s') = -\log \frac{p^\pi(s' \mid s)}{p^{\pi_\beta}(s' \mid s)}. \tag{9}$$

题目要求写成 $b(s, a)$，于是对环境转移取期望：

$$\boxed{b(s,a) = \mathbb{E}_{s'\sim p(\cdot|s,a)} \left[-\log \frac{p^\pi(s' \mid s)}{p^{\pi_\beta}(s' \mid s)}\right].} \tag{10}$$

该 bonus 不只约束单步动作概率，而是惩罚策略诱导的状态访问分布偏移。常数归一化因子可以吸收到拉格朗日乘子 λ 中。

3 Coding 总体实现

文件	主要实现
`random_agent.py`	均匀随机动作，用作探索 baseline。
`rnd_agent.py`	predictor/target network、prediction error、归一化 intrinsic reward、RND error map。
`dqn_agent.py`	epsilon-greedy、DQN/Double DQN TD target、critic update、target network。
`cql_agent.py`	在 TD loss 上加入 conservative regularizer。
`awac_agent.py`	actor policy、AWAC critic backup、advantage-weighted actor update。
`iql_agent.py`	IQL Q/V、expectile loss、implicit backup、actor advantage。
`run_hw5_finetune.py`	离线数据加载、offline/online 切换、在线 transition 插入与持续训练。

表 2: 本次作业涉及的核心文件。

4 Exploration 实验

4.1 Random Policy

4.1.1 题目要求与实现

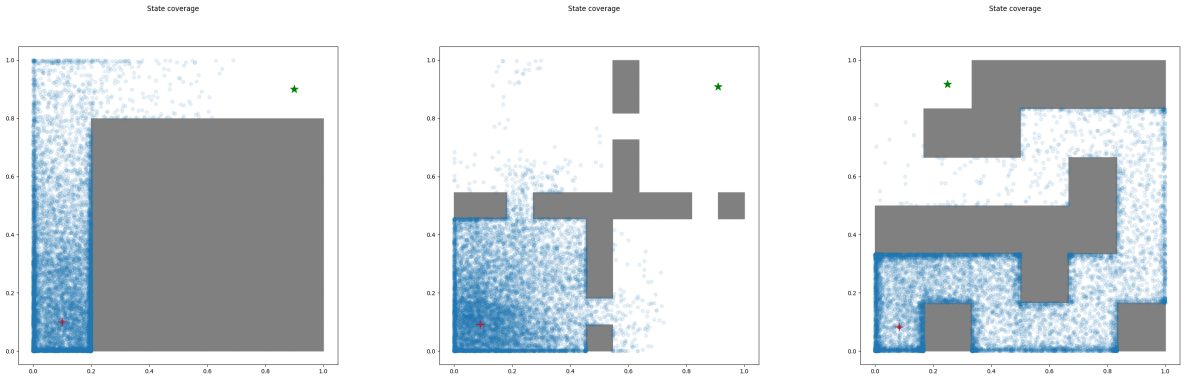
RandomAgent 不依赖 observation，也没有可训练参数，只需从离散动作空间均匀采样。

```python
class RandomAgent:
    def __init__(self, observation_shape, num_actions):
        super().__init__()
        self.num_actions = num_actions

    def get_action(self, *args, **kwargs):
        # 在 [0, num_actions) 中均匀采样动作。
        return np.random.randint(self.num_actions)

    def update(self, *args, **kwargs):
        # 随机策略没有参数需要更新。
        return {}
```

4.1.2 实验图与分析



(a) Easy random **(b)** Medium random **(c)** Hard random

图 1: 随机策略在三个 Pointmass 环境中的最终状态覆盖。

Easy 环境通道较简单，随机策略仍能覆盖一部分可达区域；Medium 和 Hard 中，随机动作容易在起点附近或局部区域徘徊。Hard 环境需要依次穿过多个狭窄通道，随机策略很难稳定到达右上角目标区域。因此随机策略适合作为 baseline，但收集高质量离线数据的效率较低。

4.2 Random Network Distillation

4.2.1 算法原理

RND 包含固定随机 target network f_θ^* 和可训练 predictor network $\hat{f}_\phi$：

$$\phi^* = \arg\min_\phi \mathbb{E}_{s' \sim \mathcal{D}} \left[\left\| \hat{f}_\phi(s') - f_\theta^*(s') \right\|_2^2 \right]. \tag{11}$$

已访问状态的 predictor error 会逐渐下降，少见状态保持较高 error，因此可以把该 error 作为 intrinsic reward。

4.2.2 RND predictor 与 intrinsic reward

```python
def update_rnd(self, obs: torch.Tensor) -> float:
    # predictor 对已访问状态进行预测。
    predictions = self.rnd_net(obs)

    # target network 固定，不参与反向传播。
    with torch.no_grad():
        targets = self.rnd_target_net(obs)

    # predictor 拟合固定随机 target。
    loss = torch.mean((predictions - targets) ** 2)

    self.rnd_optimizer.zero_grad()
    loss.backward()
    self.rnd_optimizer.step()
    return loss.item()
```

```python
def update(self, observations, actions, rewards,
           next_observations, dones, step):
    with torch.no_grad():
        # 新颖性奖励基于“到达的下一个状态”。
        predictions = self.rnd_net(next_observations)
        targets = self.rnd_target_net(next_observations)

        # 每个样本的均方 prediction error。
        rnd_error = torch.mean(
            (predictions - targets) ** 2, dim=-1
        )

        # batch 内标准化，减小不同阶段尺度差异。
        rnd_bonus = (
            rnd_error - rnd_error.mean()
        ) / (rnd_error.std() + 1e-8)

        rewards = rewards + self.rnd_weight * rnd_bonus

    # 继承 DQN critic，用 intrinsic reward 学 exploration value。
    metrics = super().update(
        observations, actions, rewards,
        next_observations, dones, step
    )

    # predictor 学习本轮刚使用的新状态。
    metrics["rnd_loss"] = self.update_rnd(next_observations)
```

```
28        metrics["rnd_error"] = rnd_error.mean().item()
29        metrics["rnd_bonus"] = rnd_bonus.mean().item()
30        return metrics
```

4.2.3 当前代码中的 exploration critic 等价实现

理论描述常把 RND 写成 exploitation critic Q_R 与 exploration critic Q_E。当前实现没有同时维护两个命名不同的 critic，但探索脚本默认将环境 reward 置零：

$$r_{\text{input}} = 0 + r_{\text{RND}}. \tag{12}$$

因此 RNDAgent 继承的 DQN critic 实际仅学习 intrinsic return：

$$Q_E(s,a) \leftarrow r_{\text{RND}} + \gamma \max_{a'} Q_E^{\text{target}}(s',a'). \tag{13}$$

当 $\epsilon = 0$ 时，`DQNAgent.get_action` 执行：

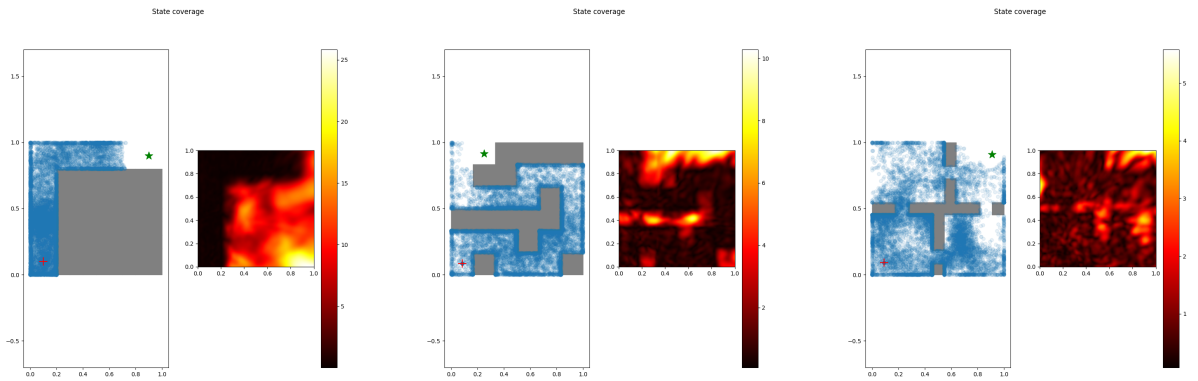
$$a_t = \arg\max_a Q_E(s_t,a), \tag{14}$$

其功能等价于题目所说的 ArgmaxPolicy。后续 exploitation value 则在 Offline RL 阶段由 DQN/CQL/AWAC/IQL 单独学习。

$$Q_E(s,a) = \mathbb{E}\left[\sum_t \gamma^t r_t^{\text{RND}}\right]$$

当前标准化会减去 batch 均值，因此日志中的平均 `rnd_bonus` 接近零是正常现象。真正决定探索方向的是样本之间的相对差异，以及 RND error map 是否在已访问区域下降、状态覆盖是否扩大。

4.2.4 实验结果



(a) Easy RND **(b)** Medium RND **(c)** Hard RND

图 2: RND 在三个环境中的状态覆盖与 error map。左侧为访问状态，右侧为 RND prediction error。

与随机策略相比，RND 在 Medium 和 Hard 环境中能主动进入更多未访问区域。已频繁访问区域的 prediction error 逐渐降低，而未访问或不可达区域保持较高 error，说明 predictor/target 机制与 exploration bonus 均正常工作。

5 Offline RL：DQN 与 CQL

5.1 DQN / Double DQN

5.1.1 动作选择

```python
def get_action(self, observation, epsilon: float = 0.0) -> int:
    # epsilon-greedy: 以 epsilon 概率随机探索。
    if np.random.rand() < epsilon:
        return int(np.random.randint(self.num_actions))

    observation = ptu.from_numpy(
        np.asarray(observation)
    )[None]

    with torch.no_grad():
        qa_values = self.critic(observation)
        action = torch.argmax(qa_values, dim=1)

    return ptu.to_numpy(action).squeeze(0).item()
```

5.1.2 critic loss

普通 DQN 使用 target critic 同时选动作和估值；Double DQN 使用 online critic 选动作、target critic 估值，从而缓解最大化偏差。

```python
def compute_critic_loss(self, obs, action, reward, next_obs, done):
    batch_size = obs.shape[0]
    action = action.long()

    qa_values = self.critic(obs)
    q_values = torch.gather(
        qa_values, dim=1, index=action.unsqueeze(1)
    ).squeeze(1)

    with torch.no_grad():
        next_qa_values = self.target_critic(next_obs)

        if self.use_double_q:
            # online network 选动作。
            next_action = torch.argmax(
                self.critic(next_obs), dim=1
            )
            # target network 对选出的动作估值。
            next_q_values = torch.gather(
                next_qa_values,
                dim=1,
                index=next_action.unsqueeze(1),
            ).squeeze(1)
        else:
            next_q_values = torch.max(
                next_qa_values, dim=1
            ).values

        target_values = reward + self.discount * (
            1.0 - done.float()
        ) * next_q_values

    loss = self.critic_loss(q_values, target_values)
    return loss, {
        "critic_loss": loss.item(),
        "q_values": q_values.mean().item(),
        "target_values": target_values.mean().item(),
    }, {
```

```
        "qa_values": qa_values,
        "q_values": q_values,
        "target_values": target_values,
    }
```

5.2 Conservative Q-Learning

CQL 在标准 TD loss 上加入：

$$\mathcal{L}_{\text{CQL}} = \mathbb{E}_s \left[\log \sum_a \exp Q(s,a) - Q(s, a_{\text{data}}) \right], \tag{15}$$

总损失为：

$$\mathcal{L} = \mathcal{L}_{\text{TD}} + \alpha \mathcal{L}_{\text{CQL}}. \tag{16}$$

```python
def compute_critic_loss(self, obs, action, reward, next_obs, done):
    # 先计算标准 DQN TD loss。
    loss, metrics, variables = super().compute_critic_loss(
        obs, action, reward, next_obs, done
    )

    qa_values = variables["qa_values"]
    q_values = variables["q_values"]

    # temperature * logsumexp(Q / temperature)
    logsumexp_q = self.cql_temperature * torch.logsumexp(
        qa_values / self.cql_temperature,
        dim=1,
    )

    cql_loss = torch.mean(logsumexp_q - q_values)
    loss = loss + self.cql_alpha * cql_loss

    metrics["cql_loss"] = cql_loss.item()
    metrics["cql_alpha"] = self.cql_alpha
    metrics["total_critic_loss"] = loss.item()
    return loss, metrics, variables
```

5.3 CQL α sweep 实验

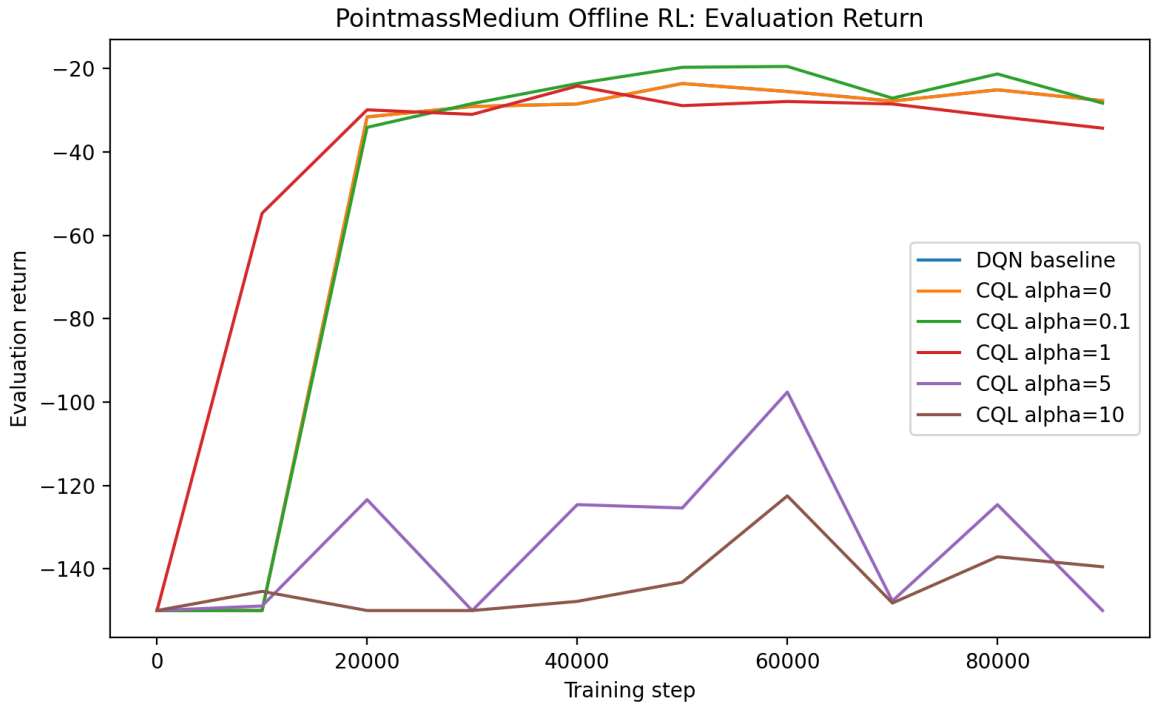
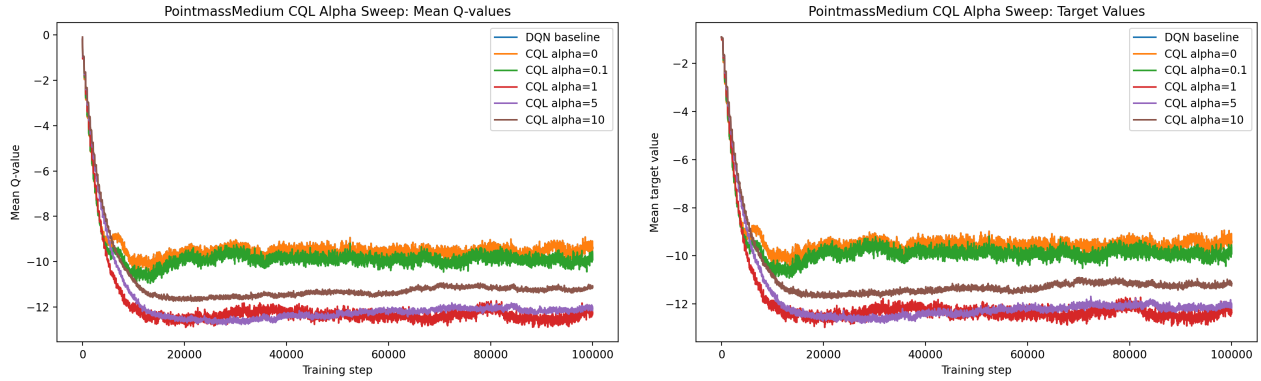


图 3: PointmassMedium-v0 上 DQN baseline 与 CQL $\alpha \in \{0, 0.1, 1, 5, 10\}$ 的 evaluation return。



(a) Mean Q-values **(b)** Mean target values

图 4: 不同 CQL α 对 Q-value 与 target value 的影响。

$\alpha = 0$ 与 $\alpha = 0.1$ 的 evaluation return 最好，其中 $\alpha = 0.1$ 更稳定；$\alpha = 1$ 仍能学习，但略弱；$\alpha = 5$ 与 $\alpha = 10$ 过度压低 Q-value，使策略长期保持在很差的回报区间。因此 CQL 的保守性不是越强越好，$\alpha = 0.1$ 在本实验中是较合理的折中。

6 Policy Constraint Methods：AWAC 与 IQL

6.1 AWAC

AWAC 使用 advantage-weighted behavior cloning：

$$\max_{\theta} \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\log \pi_\theta(a \mid s) \exp\left(\frac{A(s,a)}{\lambda} \right) \right]. \tag{17}$$

它不是平均模仿所有数据动作，而是更强地模仿 critic 认为 advantage 较高的动作。

6.1.1 AWAC critic backup

AWAC 的下一状态 target 使用当前 actor 分布下的期望：

$$y = r + \gamma(1-d)\mathbb{E}_{a'\sim\pi(\cdot|s')} Q_{\text{target}}(s', a'). \tag{18}$$

```python
def compute_critic_loss(self, observations, actions,
                        rewards, next_observations, dones):
    actions = actions.long()

    with torch.no_grad():
        next_action_dist = self._get_action_dist(next_observations)
        next_action_probs = next_action_dist.probs
        next_qa_values = self.target_critic(next_observations)

        # E_{a'~pi}[Q_target(s', a')]
        next_qs = torch.sum(
            next_action_probs * next_qa_values, dim=1
        )
        target_values = rewards + self.discount * (
            1.0 - dones.float()
        ) * next_qs

    qa_values = self.critic(observations)
    q_values = torch.gather(
        qa_values, dim=1, index=actions.unsqueeze(1)
    ).squeeze(1)

    loss = self.critic_loss(q_values, target_values)
    return loss, {
        "critic_loss": loss.item(),
        "q_values": q_values.mean().item(),
        "target_values": target_values.mean().item(),
    }, {"qa_values": qa_values, "q_values": q_values}
```

6.1.2 AWAC advantage 与 actor update

$$A(s, a_{\text{data}}) = Q(s, a_{\text{data}}) - \sum_a \pi(a \mid s) Q(s, a). \tag{19}$$

```python
def compute_advantage(self, observations, actions, action_dist=None):
    actions = actions.long()
    if action_dist is None:
        action_dist = self._get_action_dist(observations)

    with torch.no_grad():
        qa_values = self.critic(observations)
```

```
        q_values = torch.gather(
            qa_values, dim=1, index=actions.unsqueeze(1)
        ).squeeze(1)
        values = torch.sum(
            action_dist.probs * qa_values, dim=1
        )
        advantages = q_values - values
    return advantages


def update_actor(self, observations, actions):
    actions = actions.long()
    action_dist = self._get_action_dist(observations)
    log_probs = action_dist.log_prob(actions)

    advantages = self.compute_advantage(
        observations, actions, action_dist
    )
    weights = torch.exp(advantages / self.temperature)
    weights = torch.clamp(weights, max=100.0).detach()

    loss = -(log_probs * weights).mean()
    self.actor_optimizer.zero_grad()
    loss.backward()
    self.actor_optimizer.step()
    return loss.item()
```

6.2 Implicit Q-Learning

IQL 不显式对数据集外动作做 Bellman backup，而是学习单独的 value function：

$$\mathcal{L}_V(\phi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_2^\tau(Q_\theta(s,a) - V_\phi(s))\right], \tag{20}$$

$$\mathcal{L}_Q(\theta) = \mathbb{E}_{(s,a,s')\sim\mathcal{D}}\left[r + \gamma V_{\bar{\phi}}(s') - Q_\theta(s,a)\right]^2. \tag{21}$$

6.2.1 Expectile loss

```
@staticmethod
def iql_expectile_loss(expectile, vs, target_qs):
    diff = target_qs - vs
    weights = torch.where(
        diff > 0,
        torch.full_like(diff, expectile),
        torch.full_like(diff, 1.0 - expectile),
    )
    return torch.mean(weights * diff.pow(2))
```

6.2.2 Q-function 更新

```
def update_q(self, observations, actions, rewards,
             next_observations, dones):
    actions = actions.long()

    with torch.no_grad():
        next_vs = self._squeeze_value(
            self.target_value_critic(next_observations)
        )
        target_values = rewards + self.discount * (
            1.0 - dones.float()
        ) * next_vs

    qa_values = self.critic(observations)
```

```
    q_values = torch.gather(
        qa_values, dim=1, index=actions.unsqueeze(1)
    ).squeeze(1)

    loss = self.critic_loss(q_values, target_values)
    self.critic_optimizer.zero_grad()
    loss.backward()
    grad_norm = torch.nn.utils.clip_grad.clip_grad_norm_(
        self.critic.parameters(),
        self.clip_grad_norm or float("inf")
    )
    self.critic_optimizer.step()

    return {
        "q_loss": loss.item(),
        "q_values": q_values.mean().item(),
        "q_target_values": target_values.mean().item(),
        "q_grad_norm": grad_norm.item(),
    }
```

6.2.3 Value function 更新

```
def update_v(self, observations, actions):
    actions = actions.long()

    with torch.no_grad():
        qa_values = self.target_critic(observations)
        target_qs = torch.gather(
            qa_values, dim=1, index=actions.unsqueeze(1)
        ).squeeze(1)

    vs = self._squeeze_value(
        self.value_critic(observations)
    )
    loss = self.iql_expectile_loss(
        self.expectile, vs, target_qs
    )

    self.value_critic_optimizer.zero_grad()
    loss.backward()
    self.value_critic_optimizer.step()

    return {
        "v_loss": loss.item(),
        "vs": vs.mean().item(),
        "v_target_values": target_qs.mean().item(),
    }
```

IQL actor 继承 AWAC 的加权最大似然形式，但 advantage 改为：

$$A_{\mathrm{IQL}}(s,a) = Q(s,a) - V(s). \tag{22}$$

完整更新顺序是：更新 Q、更新 V、更新 actor，并按周期同步 Q target 与 V target。

6.3 AWAC 与 IQL 实验对比

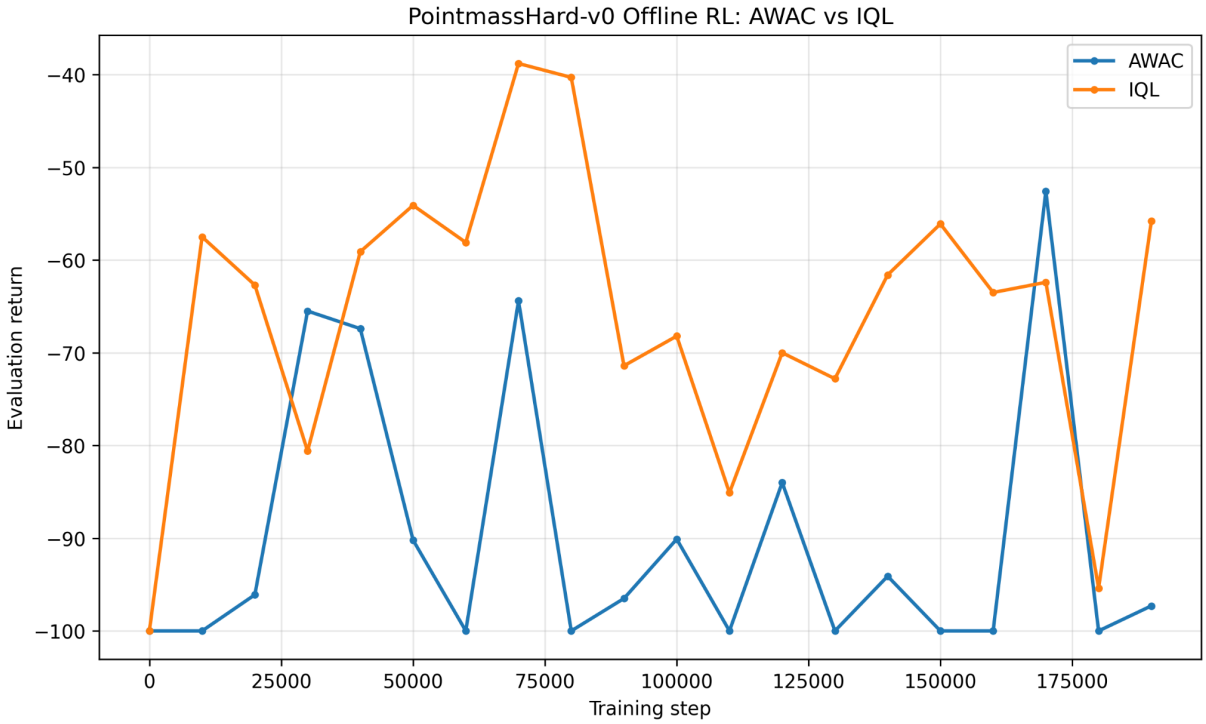


图 5: PointmassHard-v0 上 AWAC 与 IQL 的 offline RL evaluation return。

方法	最佳回报	最佳 step	最终回报	最终 step
AWAC	−52.6	170000	−97.3	190000
IQL	−38.8	70000	−55.8	190000

表 3: AWAC 与 IQL 的真实实验结果。

IQL 的最佳回报比 AWAC 高 13.8，最终回报比 AWAC 高 41.5。AWAC 多次回落至接近 −100，说明 actor 与 critic 相互依赖时容易发生训练退化；IQL 通过 $V(s')$ 做 implicit backup，不显式查询任意 policy 产生的 OOD action，因此在当前 PointmassHard 离线数据上整体更稳定。

7 Data Ablation

7.1 实验设计

在 PointmassMedium-v0 上分别使用 RND 收集 1000、5000、10000 和 20000 个 transition，随后固定 CQL $\alpha = 0.1$，以相同训练配置分别训练 CQL agent。作业要求报告 evaluation performance；本报告同时给出 mean Q-value，以分析价值估计与真实策略表现是否一致。

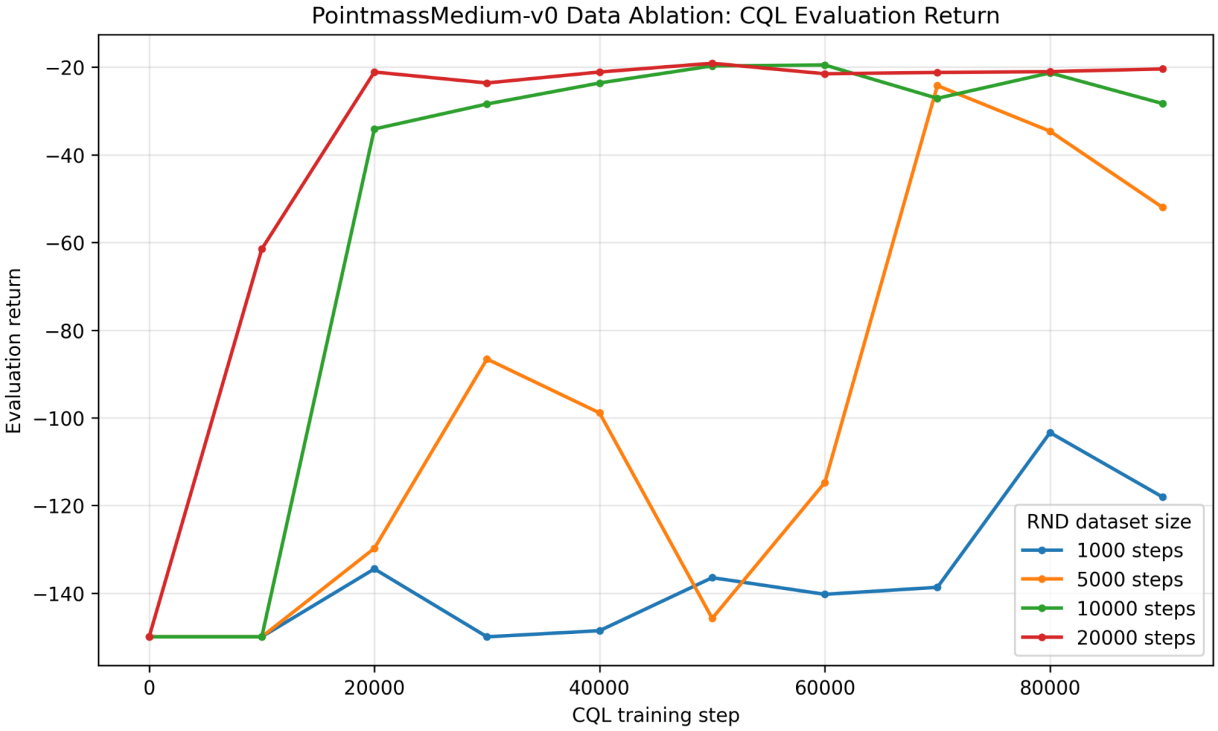


图 6: 不同 RND 数据规模对应的 CQL evaluation return。

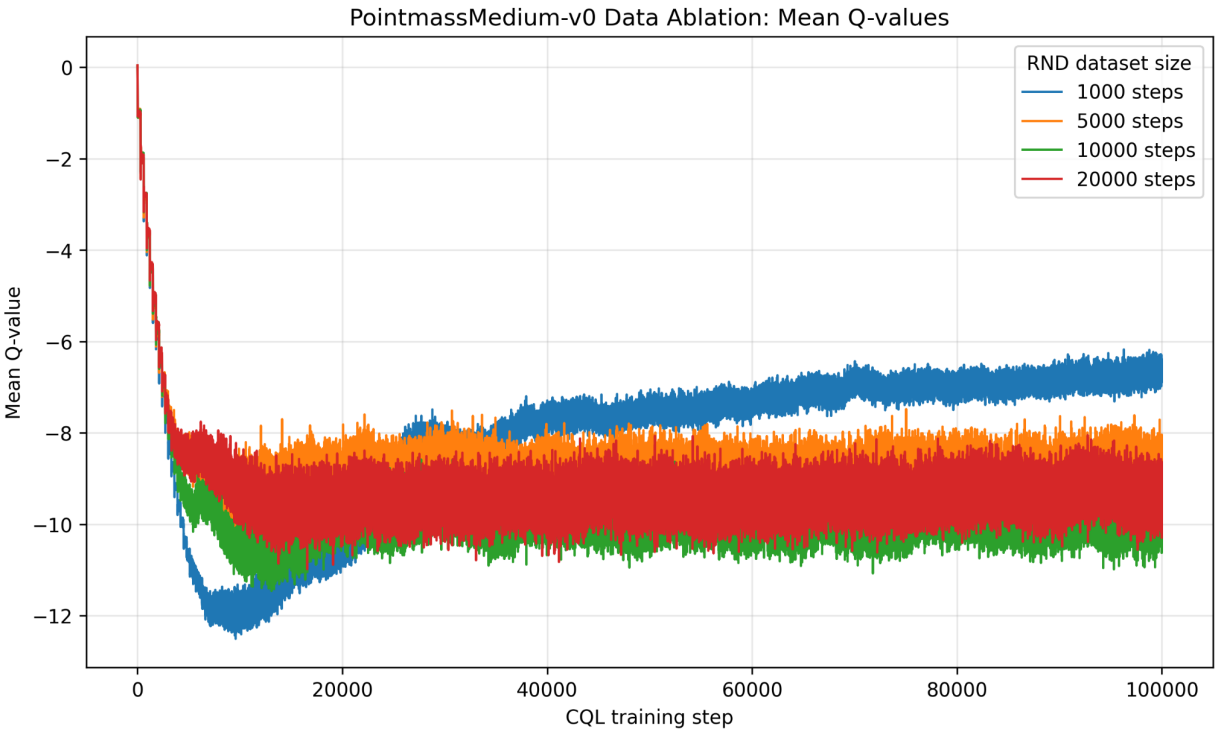


图 7: 不同 RND 数据规模对应的 CQL mean Q-value。

数据量	最佳回报	最终回报	后 5 次均值	后 5 次标准差
1000	−103.4	−118.1	−127.40	14.42
5000	−24.2	−52.0	−74.28	47.61
10000	−19.5	−28.3	−23.18	3.76
20000	−19.1	−20.4	−20.64	0.85

表 4: Data Ablation 的最佳、最终与稳定性指标。

7.2 结果分析

1000-step 数据集覆盖严重不足，策略始终无法稳定完成任务。5000-step 数据集偶尔达到 −24.2，但后五次标准差高达 47.61，说明少量有效轨迹不足以维持稳定学习。10000-step 数据集已经能获得较高回报，20000-step 数据集则在最佳回报、最终回报和稳定性上综合最好。

更值得注意的是，1000-step 实验后期的 mean Q-value 反而最高，但 evaluation return 最差。这说明：

$$\text{较高的估计 Q-value} \not\Rightarrow \text{较好的真实策略性能.} \tag{23}$$

数据覆盖不足时，critic 可能对未覆盖区域产生不可靠估计。CQL 可以抑制 OOD action 的过估计，但无法补充数据集中完全缺失的成功路径。较大数据集的作用不是简单把 Q-value 推高，而是提高 Q-value 与真实 return 的一致性。

Data Ablation 支持“offline RL 的上限首先由数据决定”的结论。20000-step 数据集后五次回报均值为 −20.64、标准差仅 0.85，而 1000-step 数据集即使出现较乐观的 Q-value，真实回报仍很差。评价 offline RL 时必须同时观察 evaluation return、稳定性、状态覆盖和 Q-value，而不能只看 critic 输出。

8 Online Fine-Tuning

8.1 题目要求

在线微调流程为：先加载固定 offline dataset 并训练若干步，再切换到环境交互，同时保留原离线数据作为 replay buffer 初始化，并持续把新 transition 插入 buffer。结果图需要明确标出 online fine-tuning 的开始点。题目期望 PointmassHard-v0 上稳定达到至少约 −20。

8.2 核心实现

8.2.1 加载并扩容离线 replay buffer

```
offline_dataset_file = os.path.join(
    args.dataset_dir,
    f"{config['dataset_name']}.pkl",
)
with open(offline_dataset_file, "rb") as f:
    offline_dataset = pickle.load(f)

num_offline_steps = config["offline_steps"]
num_online_steps = (
    config["total_steps"] - num_offline_steps
```

```
)

replay_capacity = max(
    int(config.get("replay_capacity", config["total_steps"])),
    len(offline_dataset) + num_online_steps + 1,
)
replay_buffer = copy_replay_buffer_to_larger_buffer(
    dataset=offline_dataset,
    capacity=replay_capacity,
)
```

8.2.2 offline/online 阶段切换与参数调整

```
for step in tqdm.trange(config["total_steps"]):
    is_online = step >= num_offline_steps

    # online 阶段可以降低 CQL 保守强度。
    if (
        is_online
        and "online_cql_alpha" in config
        and hasattr(agent, "cql_alpha")
    ):
        agent.cql_alpha = config["online_cql_alpha"]

    # online epsilon 线性衰减。
    if is_online:
        online_step = step - num_offline_steps
        decay_steps = config.get(
            "online_epsilon_decay_steps",
            num_online_steps,
        )
        frac = min(1.0, online_step / max(1, decay_steps))
        epsilon = (
            config["online_epsilon_start"]
            + frac * (
                config["online_epsilon_end"]
                - config["online_epsilon_start"]
            )
        )
```

8.2.3 在线交互与持续训练

```
if is_online:
    action = agent.get_action(observation, epsilon)
    next_observation, reward, done, info = env.step(action)
    next_observation = np.asarray(next_observation)

    truncated = info.get("TimeLimit.truncated", False)
    replay_buffer.insert(
        observation=observation,
        action=action,
        reward=reward,
        next_observation=next_observation,
        done=done and not truncated,
    )

    observation = env.reset() if done else next_observation

batch = replay_buffer.sample(config["batch_size"])
batch = {
    k: ptu.from_numpy(v) if isinstance(v, np.ndarray) else v
    for k, v in batch.items()
}

update_info = agent.update(
```

```
    batch["observations"],
    batch["actions"],
    batch["rewards"],
    batch["next_observations"],
    batch["dones"],
    step,
)
```

8.3 实验结果

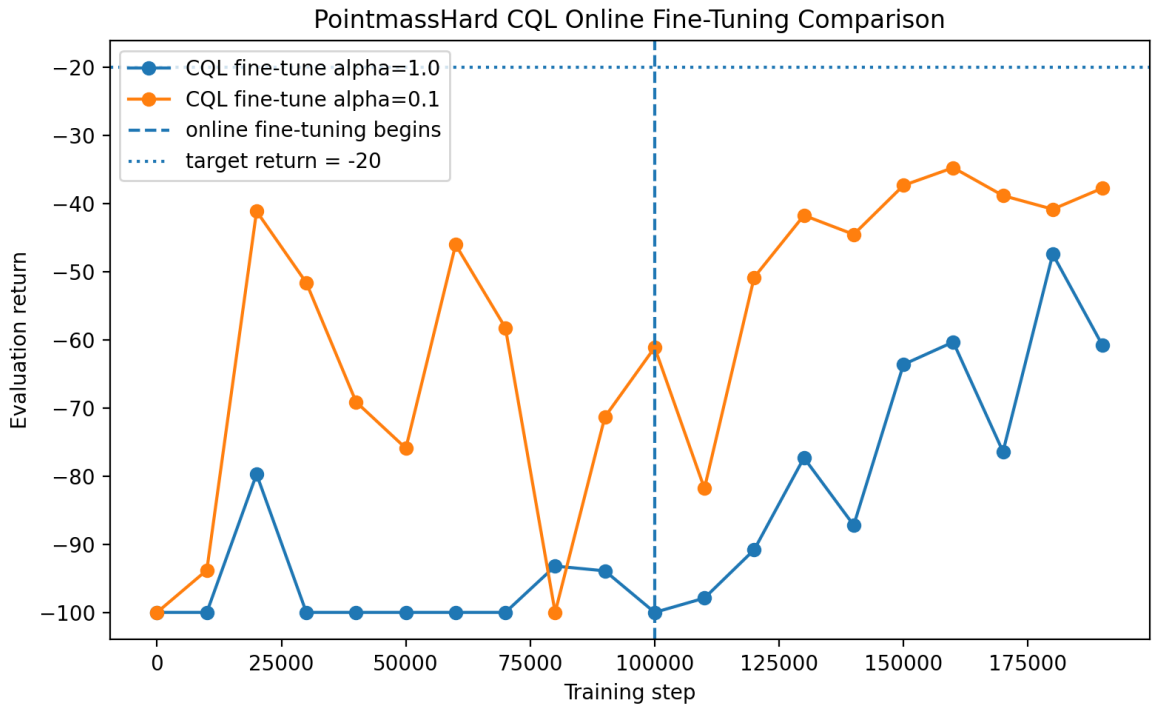


图 8: PointmassHard-v0 上 CQL online fine-tuning 对比。竖线表示 online 阶段开始，水平虚线为目标回报 −20。

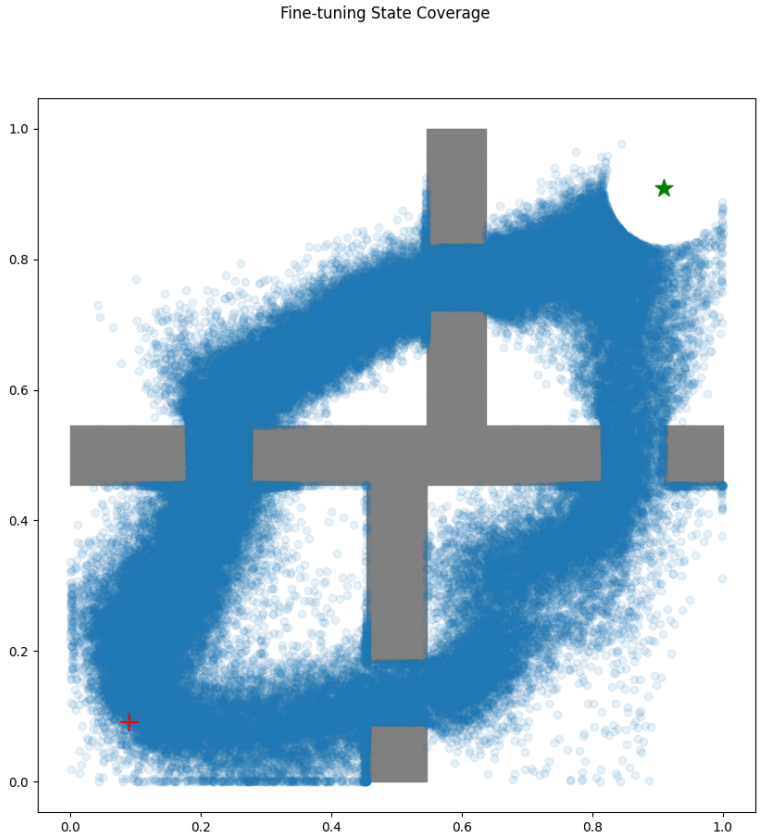


图 9: recent-buffer fast 实验的最终状态覆盖。该实验最佳 evaluation return 为 -30.8，最终为 -33.8。

Online fine-tuning 相比纯 offline training 有明显提升，较小的 online CQL α 优于持续保持较强保守性。状态覆盖图表明 agent 已形成从左下起点通向右上目标附近的主要路径，但轨迹仍较分散，说明到达目标的行为尚不够短、稳定和可重复。

8.4 未稳定达到 -20 的原因

1. **初始数据覆盖不足：** 原始 PointmassHard RND 数据约 10000 条 transition，可能没有充分覆盖所有狭窄通道和高质量成功轨迹。
2. **稀疏奖励与路径稳定性：** 策略能接近目标，但并非每次 evaluation 都稳定到达；少量偏差可能导致穿越窄门失败。
3. **CQL 保守性与适应速度的平衡：** offline 阶段需要抑制 OOD overestimation，但 online 阶段过强的保守约束会阻碍 critic 快速利用新数据。
4. **replay buffer 样本质量：** online 数据加入后，buffer 中仍保留大量失败轨迹。若高质量新数据比例不足，更新会持续受到旧失败样本影响。

9 总结与个人思考

本次作业把探索、离线训练和在线微调连接成了一个完整流程。RND 说明高质量数据并不是随机采样自然产生的，而需要主动扩大状态覆盖；CQL 说明离线价值学习必须处理数据外动作，但保守系数过大也会损伤有用动作；AWAC 与 IQL 的比较说明，显式依赖 actor 的 critic backup 更容易受到 actor 不稳定影响，而 IQL 的 implicit backup 能更直接地避开 OOD action；Data Ablation 进一步证明，算法不能替代数据覆盖，且高 Q-value 不一定意味着高真实回报。

Online fine-tuning 为策略提供了补充数据的机会，但它并不会自动解决全部问题。初始策略若太弱，新采样数据仍可能以失败轨迹为主；buffer 中旧数据比例过高，也会使新成功样本的影响被稀释。因此，在后续改进中，比单纯延长训练更值得尝试的方向包括：提高初始 RND 数据的成功轨迹比例、使用分层或优先级 replay、在 online 阶段动态调整 offline/online sampling ratio，以及对关键窄门区域设计更有针对性的探索机制。

Offline RL 的上限首先由数据覆盖决定，其次才是算法。算法可以降低 OOD overestimation、约束策略和利用少量在线数据，但不能从完全缺失的轨迹中凭空恢复最优行为。评价结果时应同时观察真实 evaluation return、方差、状态覆盖和 Q-value 一致性。

10 提交说明

最终提交应包含：

1. 本 PDF 报告，包含理论题、关键代码、全部必做实验曲线与分析；
2. `cs285/` 文件夹，保持原作业目录结构与文件名；
3. `data/` 文件夹，包含各实验 TensorBoard 日志；
4. README，写明运行环境、每张图对应的命令及生成方式；
5. 压缩包中实验目录保留题目要求的前缀和命名，且总大小满足提交平台限制。

参考文献

[1] Berkeley CS 285, Fall 2023. *Assignment 5: Exploration Strategies and Offline Reinforcement Learning.*

[2] Burda, Y. et al. *Exploration by Random Network Distillation.*

[3] Kumar, A. et al. *Conservative Q-Learning for Offline Reinforcement Learning.*

[4] Nair, A. et al. *Accelerating Online Reinforcement Learning with Offline Datasets.*

[5] Kostrikov, I. et al. *Offline Reinforcement Learning with Implicit Q-Learning.*